

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Lambda Calculus

- Lambda Expressions

- Conversions of λ -expressions

- Arithmetic in Lambda Calculus

- Logical Operations and Conditionals

- Recursion

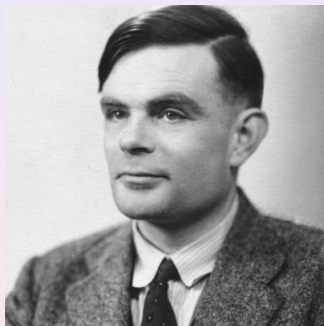
Typed lambda calculus

Semantics usings lambdas

Combinatory Logic

A little bit of history, please!

Alan Turing: Turing machine



All I need:
an infinite strip of paper + a pencil to write symbols + an eraser
+ a set of rules about what to write

A little bit of history, please!

John von Neumann: von Neumann architecture



All I need:

memory to store data *and* instructions + a processing unit + a
program counter to show me what to do next

A little bit of history, please!

Alonzo Church: Lambda Calculus



All I need:
 $\lambda.()$ and variable names

A little bit of history, please!

Haskell Curry: combinatory logic



All I need:
S and **K**

A little bit of history, please!

Kurt Gödel: recursive functions



All I need:
functions $\mathbb{N}^k \rightarrow \mathbb{N}$ + functions zero, successor, projection +
function composition + primitive recursion + minimization

Introduction

What is Lambda Calculus?

1. is a **universal**, but minimalist programming language
2. as expressive as a Turing machine, but ...
3. is based on mapping inputs into outputs, **with no side effects**, while a Turing machine relies on changing states
4. it uses only **unary** expressions
5. it was introduced in 1930s by Alonzo Church
6. believed to be able to express any computable function ("Turing-Church thesis")
7. most of the nowadays functional languages rely on it

Bibliography



R. Rojas

A Tutorial Introduction to the Lambda Calculus
FU Berlin, Germany, 2015



David Beazley

Lambda Calculus from the Ground Up – PyCon 2019



R. Smullyan

To Mock a Mockingbird
ISBN-13: 978-0394534916



Mario Román

Mikrokosmos – an interpreter for λ -calculus

λ -expressions: the basic blocks of λ -calculus

Definition (λ -expressions)

$$E = x \mid (\lambda x. E) \mid (E_1 E_2)$$

- ▶ x is a variable (e.g., $a, \dots, z$)
- ▶ $(\lambda x. E)$ is called " λ -abstraction". E.g.,
 - ▶ $(\lambda x. x)$ (aka `id`)
 - ▶ $(\lambda x. (\lambda y. x))$ (aka `first`)
- ▶ $(E_1 E_2)$ means "applying E_1 on E_2 ". E.g.,
 - ▶ $(t u)$ means "apply expression t on argument u "
 - ▶ $((\lambda p. p) q)$ means "apply the λ -expression on q "

That's all, folks: λ . () *variables*!



λ -expressions

Definition (Bound variables and body)

For the abstraction $(\lambda x.t)$, x is said to be **bound**; t is the abstraction's **body**.

Definition (Free variables)

A variable b which is not bound is called **free**.

Example (Bound and free variables)

Inside the expression $(\lambda z.(\lambda x.(b\ x)))$, x is bound and b is free.

Remark

The name of a bound variable does not matter (e.g. $(\lambda x.x\ a) \equiv (\lambda y.y\ a)$), but the name of a free variable does ($a \neq b$).

λ -expressions

Syntax simplifications

- ▶ $\ll$ collapse $\gg$ nested lambdas (but expressions still *unary*):
 $(\lambda x_1.(\lambda x_2 \dots (\lambda x_n.t) \dots)) \equiv (\lambda x_1 x_2 \dots x_n.t)$
- ▶ applications associates from left to right:
 $(\dots ((t_1 t_2) t_3) \dots t_n) \equiv (t_1 t_2 t_3 \dots t_n)$
- ▶ outer parentheses dropped for the outermost expression or if they simply enclose the body of another λ -expression:
 $(\lambda x.(x(\lambda y.(y x)))) \equiv \lambda x.x(\lambda y.y x)$
- ▶ λ 's extend to the right as much as possible: $\lambda x.yzxa$
means $\lambda x.(y z x a)$ (*it does not mean* $(\lambda x.y z x) a$)
- ▶ if, inside the λ -expression $(\lambda x.t)$, we replace every bound occurrence of x in t by u , we will write $t[u/x]$

α -conversion

Definition (Conversion)

Conversion = rule for transforming a λ -expression into another one with the same meaning.

Definition (α -conversion: rename bound variables)

$$(\lambda x.t) \equiv_{\alpha} (\lambda y.t[y/x])$$

Example

$$(\lambda x.a\ x) \equiv_{\alpha} (\lambda y.a\ y)$$

Definition (Equivalence of two λ -expressions)

Two λ -expressions are said to be equivalent if they can be transformed into identical λ -expressions using α -conversions.

β -conversion

Definition (β -conversion: apply abstractions to arguments)

$$((\lambda x.t)u) \equiv_{\beta} (t[u/x])$$

Example

$$((\lambda x.((f\ x)x))(g\ a)) \equiv_{\beta} ((f(g\ a))(g\ a))$$

Example

$$\begin{aligned} ((\lambda z.(z\ a))(\lambda x.x)) &\equiv_{\beta} ((\lambda x.x)a) \\ &\equiv_{\beta} a \end{aligned}$$

β -conversion

Example

$$\begin{aligned}((\lambda x y.y)a)b &\equiv_{\beta} (\lambda y.y[a/x])b \\ &\equiv (\lambda y.y)b \\ &\equiv_{\beta} y[b/y] \\ &\equiv b\end{aligned}$$

Example (Danger! Accidentally binding free variables)

$$(\lambda x y.y x)t u \equiv (\lambda y.y t)u \equiv u t$$

$$\begin{aligned}\text{Wrong: } (\lambda \textcolor{red}{x} y.y \textcolor{red}{x}) \textcolor{red}{y} b &\equiv (\lambda y.y y) b \\ &\equiv b b\end{aligned}$$

$$\begin{aligned}\text{Right: } (\lambda x y.y x)y b &\equiv (\lambda x z.z x)y b \\ &\equiv (\lambda z.z y)b \\ &\equiv b y\end{aligned}$$

η -conversion

Definition (η -conversion: add/drop abstraction)

$$(\lambda x. f x) \equiv_{\eta} f$$

Example

$$(\lambda s. (\lambda z. z y) s) \equiv_{\eta} (\lambda z. z y)$$

Normal form

Definition (Reduction)

A reduction step of t into u (denoted here by $t \equiv u$) transforms the λ -expression t into an equivalent one u by applying a β -conversion to any subexpression of t .

Definition (Normal form of a λ -expression)

A λ -expression is said to be in the **normal form** if no further reduction steps could be applied to it.

Example

$$(\lambda x. a \ x)((\lambda y. b \ y)c) \equiv a((\lambda y. b \ y)c) \equiv a(b \ c)$$

Normal form

Church-Rosser theorem

Two sequences of reductions applied to a λ -expression will always result in equivalent normal forms. Thus the normal form does not depend on the order the reductions are performed.

Example

$$(\lambda x. a x)((\lambda y. b y)c) \equiv a((\lambda y. b y)c) \equiv a(b c)$$

$$(\lambda x. a x)((\lambda y. b y)c) \equiv (\lambda x. a x)(b c) \equiv a(b c)$$

Normal form

Example (Some λ -expressions don't have a normal form)

$$(\lambda x.x\ x)(\lambda x.x\ x) \equiv (\lambda x.x\ x)(\lambda x.x\ x) \dots$$

Example (A "call-by-name" like substitution)

$$(\lambda y.a)((\lambda x.x\ x)(\lambda x.x\ x)) \equiv a$$

This type of substitution always finds the normal form, provided that such a form actually exists.

Example (A "call-by-value" like substitution)

$$(\lambda y.a)((\lambda x.x\ x)(\lambda x.x\ x)) \equiv (\lambda y.a)((\lambda x.x\ x)(\lambda x.x\ x)) \equiv \dots$$

This type of substitution might never end, thus failing to find the normal form, even if such a form does exist.

Redex

Definition (Redex)

Redex (**Red**ucible **ex**pression) = a λ -expression over which the β -conversion could be immediately performed.

Example

$\lambda x.(\lambda y.b\ y)c$ is not a redex, but its body $(\lambda y.b\ y)c$ is

Which redex to reduce first, if several are available?

1. the left-most one
2. the right-most one
3. the left-most one, if it's not in the body of a λ -abstraction

Arithmetic using λ -expressions

Question: can we represent numbers in λ -calculus?

YES

Definition (Natural numbers (Church numerals))

$0 := \lambda s z. z$

$1 := \lambda s z. s(z)$

$2 := \lambda s z. s(s(z))$

...

Remark

A number n defined in this way represents rather "argument #1 applied n times over argument #2" or "s applied n times to z "

Arithmetic using λ -expressions

Example (Can we do that in Python?)

```
>>> ONE = lambda f: lambda x: f(x)
>>> TWO = lambda f: lambda x: f(f(x))
>>> THREE = lambda f: lambda x: f(f(f(x)))
>>> FOUR = lambda f: lambda x: f(f(f(f(x))))
```

```
#this is an alien in the lambda world!
```

```
>>> def f(x):
...     return 'glory ' + x
```

```
>>> TWO (f) ('hallelujah')
```

```
'glory glory hallelujah'
```

Arithmetic using λ -expressions

Definition (Getting the successor of a natural number)

$\text{SUCC} := \lambda wyx. y(wyx)$

Example (Successor of 0)

$$\begin{aligned}\text{SUCC } 0 &= (\lambda wyx. y(wyx))0 \\ &= (\lambda wyx. y(wyx))(\lambda sz. z) \\ &= \lambda yx. y((\lambda sz. z)yx) \\ &= \lambda yx. y((\lambda z. z)x) \\ &= \lambda yx. y(x) \\ &= 1\end{aligned}$$

Example

Show that $\text{SUCC } 1 = 2$ and $\text{SUCC } 2 = 3$

Arithmetic using λ -expressions

Definition (Adding numbers: repeat SUCC)

$\text{PLUS} := \lambda mn.m \text{ SUCC } n$

Example (1+2)

$$\begin{aligned}\text{PLUS } 1 \ 2 &= (\lambda mn.m \text{ SUCC } n) \ 1 \ 2 \\ &= 1 \ \text{SUCC } 2 \\ &= (\lambda fx.fx) \ \text{SUCC } 2 \\ &= \text{SUCC } 2 \\ &= 3\end{aligned}$$

Definition (Multiplying numbers: repeat addition)

$\text{MULT} := \lambda mn.m \ (\text{PLUS } n) \ 0$

Logic using λ -expressions

Question: can we do logical operations in λ -calculus?

YES

Definition (Logical values)

$\mathcal{T} := \lambda xy.x$

$\mathcal{F} := \lambda xy.y$

Definition (Logical AND)

$\wedge := \lambda xy.xy(\lambda uv.v) \equiv \lambda xy.xy\mathcal{F}$

Remark

If the first argument passed to $\wedge$ is $\mathcal{F}$, the whole expression will be $\mathcal{F}$ since the λ -expression of $\mathcal{F}$ selects its second argument.

Logic using λ -expressions

Example

$$\begin{aligned}\wedge \mathcal{T} \mathcal{F} &\equiv (\lambda xy.xy\mathcal{F})\mathcal{T}\mathcal{F} \\ &\equiv \mathcal{T}\mathcal{F}\mathcal{F} \\ &\equiv (\lambda xy.x)\mathcal{F}\mathcal{F} \\ &\equiv \mathcal{F}\end{aligned}$$

Definition (Logical OR and NOT)

$$\begin{aligned}\vee &:= \lambda xy.x(\lambda uv.u)y \equiv \lambda xy.x\mathcal{T}y \\ \neg &:= \lambda x.x(\lambda uv.v)(\lambda ab.a) \equiv \lambda x.x\mathcal{F}\mathcal{T}\end{aligned}$$

Example

Show that $\neg \mathcal{F} \equiv \mathcal{T}$

Logic using λ -expressions

Definition (Conditional expression if-then-else)

$\text{COND} := \lambda ite. ite$

Example

$$\begin{aligned}\text{COND } \mathcal{T} \ a \ b &\equiv (\lambda ite. ite) \mathcal{T} \ a \ b \\ &\equiv \mathcal{T} \ a \ b \\ &\equiv (\lambda xy. x) \ a \ b \\ &\equiv a\end{aligned}$$

Logic using λ -expressions

Definition (Test whether a number is zero or not)

$$\text{isZ} := \lambda x. x\mathcal{F}\neg\mathcal{F}$$

Example

$$\begin{aligned}\text{isZ } 0 &\equiv (\lambda x. x\mathcal{F}\neg\mathcal{F})0 \\ &\equiv 0\mathcal{F}\neg\mathcal{F} \\ &\equiv \neg\mathcal{F} \equiv \dots \equiv \mathcal{T}\end{aligned}$$

Example

$$\begin{aligned}\text{isZ } 1 &\equiv (\lambda x. x\mathcal{F}\neg\mathcal{F})1 \\ &\equiv 1\mathcal{F}\neg\mathcal{F} \\ &\equiv \mathcal{F}(\neg)\mathcal{F} \\ &\equiv \mathcal{F}\end{aligned}$$

Defining pairs

Question: can we represent pairs in λ -calculus?

YES

Definition (Pairs)

$\text{PAIR} := \lambda xyf.fxy$

$\text{FST} := \lambda p.p\mathcal{T}$

Example

$$\begin{aligned}\text{FST } (\text{PAIR } a \ b) &\equiv (\lambda p.p \ \mathcal{T})(\text{PAIR } a \ b) \\ &\equiv (\text{PAIR } a \ b) \ \mathcal{T} \\ &\equiv (\lambda f.fab)\mathcal{T} \\ &\equiv \mathcal{T} \ a \ b \\ &\equiv a\end{aligned}$$

Defining pairs

Definition (Pairs)

$\text{SND} := \lambda p. p\mathcal{F}$

$\text{NIL} := \lambda x. \mathcal{T}$ (alternative: $\lambda pq. q$)

Remark

Once we have pairs, we can implement lists, then trees etc.

Recursivity and λ -expressions

Question: can we model recursive calls in λ -calculus?

YES

Definition (Shift-and-Increment: $(m, n) \mapsto (n, \text{SUCC } n)$)

$\Phi := \lambda x. \text{PAIR } (\text{SND } x) (\text{SUCC } (\text{SND } x))$

Example

$\Phi (\text{PAIR } m \ n)$
 $\equiv (\lambda x. \text{PAIR } (\text{SND } x) (\text{SUCC } (\text{SND } x))) (\text{PAIR } m \ n)$
 $\equiv \text{PAIR } (\text{SND } (\text{PAIR } m \ n)) (\text{SUCC } (\text{SND } (\text{PAIR } m \ n)))$
 $\equiv \text{PAIR } n (\text{SUCC } n)$

Recursivity and λ -expressions

Definition (Getting the predecessor of a natural number)

$\text{PRED} := \lambda n. \text{FST } (n \ \Phi \ (\text{PAIR } 0 \ 0))$

Example (Predecessor of 3)

$$\begin{aligned}\text{PRED } 3 &\equiv (\lambda n. \text{FST } (n \ \Phi \ (\text{PAIR } 0 \ 0)))3 \\ &\equiv \text{FST } (3 \ \Phi \ (\text{PAIR } 0 \ 0)) \\ &\equiv (3 \ \Phi \ (\text{PAIR } 0 \ 0)) \ \mathcal{T} \\ &\equiv (\Phi \ (\Phi \ (\Phi \ (\text{PAIR } 0 \ 0)))) \ \mathcal{T} \\ &\equiv (\text{PAIR } 2 \ 3) \ \mathcal{T} \\ &\equiv ((\lambda xyf. fxy) \ 2 \ 3) \ \mathcal{T} \\ &\equiv \mathcal{T} \ 2 \ 3 \\ &\equiv 2\end{aligned}$$

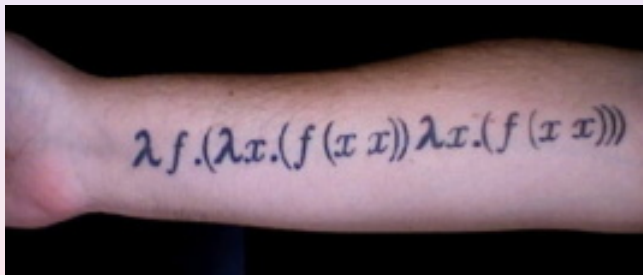
Example

Show that $\text{PRED } 2 = 1$ and $\text{PRED } 1 = 0$

Recursivity and λ -expressions

Definition (The fixed-point combinator Y)

$$Y := \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$



Recursivity and λ -expressions

Definition (The fixed-point combinator Y)

$$Y := \lambda y.(\lambda x.y(xx))(\lambda x.y(xx))$$

Example (Apply Y on an arbitrary R: "self-regeneration")

$$\begin{aligned} YR &\equiv \lambda y.(\lambda x.y(xx))(\lambda x.y(xx))R \\ &\equiv (\lambda x.R(xx))(\lambda x.R(xx)) \\ &\equiv R((\lambda x.R(xx))(\lambda x.R(xx))) \\ &\equiv R(YR) \end{aligned}$$

Remark

Y works like a "machine" which finds a fixed-point of R

Recursivity and λ -expressions

Remark

We have: $YR \equiv R(YR)$.

This looks pretty much like a recursive call!

When Y is applied on R , R will call itself on (YR) . Subsequent equivalent forms appear when R is expanded according to its definition. It will be the responsibility of R to cut off the recursive sequence at some moment by providing a stopping condition.

Recursivity and λ -expressions

Example (Factorial of n : a recursive formula)

$n! = n * (n - 1)!$ if $n \geq 1$; $0! = 1$

Example (Computing $n!$ in λ -calculus)

$MULT := \lambda m n. m (PLUS\ n)\ 0$

$FACTO := \lambda r n. isZ\ n\ 1\ (MULT\ n\ (r\ (PRED\ n)))$

What does FACTO do?

FACTO expects arguments r and n . If n is 0, FACTO returns 1; otherwise it calls r on n 's predecessor and multiplies the result by n .

Recursivity and λ -expressions

Example (Computing 3! step-by-step)

$Y \text{ FACTO } 3 \equiv \text{FACTO } (Y \text{ FACTO}) 3$
 $\equiv (\lambda r n. \text{isZ } n \ 1 \ (\text{MULT } n \ (r \ (\text{PRED } n)))) (Y \text{ FACTO}) 3$
 $\equiv \text{isZ } 3 \ 1 \ (\text{MULT } 3 \ (\text{Y FACTO } (\text{PRED } 3)))$

FACTO expands, then calls itself from within

$\equiv \text{MULT } 3 \ (Y \text{ FACTO } (\text{PRED } 3))$
 $\equiv \text{MULT } 3 \ (Y \text{ FACTO } 2)$

$Y \text{ FACTO } 2 \equiv \text{MULT } 2 \ (Y \text{ FACTO } 1)$

$Y \text{ FACTO } 1 \equiv \text{MULT } 1 \ (Y \text{ FACTO } 0)$

$Y \text{ FACTO } 0 \equiv \text{FACTO } (Y \text{ FACTO}) 0$
 $\equiv \text{isZ } 0 \ 1 \ (\text{MULT } 0 \ (Y \text{ FACTO } (\text{PRED } 0)))$
 $\equiv 1$

Thus $Y \text{ FACTO } 3 \equiv \text{MULT } 3 \ (\text{MULT } 2 \ (\text{MULT } 1 \ 1))$

de Bruijn index: no names for variables

*What's in a name? that which we call a rose
By any other name would smell as sweet*

Shakespeare

Definition (de Bruijn index)

de Bruijn index of a variable x could be seen as the minimum number of λ 's we need to pass to get outside the one binding x .

Example

$\lambda x. \lambda y. (y\ x)$ is written as $\lambda\lambda(1\ 2)$. Indices: 1 for y , 2 for x

de Bruijn index: no names for variables

Example (writing 1)

$\lambda x. \lambda y. x$ is written as $\lambda\lambda 2$

Example (writing 2)

$\lambda x. \lambda y. \lambda z. (x\ z\ (y\ z))$ is written as $\lambda\lambda\lambda((3\ 1)\ (2\ 1))$

Example (writing 3)

$\lambda x. (\lambda y. (\lambda z. z\ z)\ y\ (\lambda z. z\ y))$ is written as $\lambda\lambda((\lambda(1\ 1)\ 1)\ \lambda(1\ 2))$

Example (application)

$(\lambda\lambda 2\ \lambda 1) \equiv \lambda\lambda 1$

Typed lambda calculus: motivation

Example (Two expressions for $\wedge$)

LAND $:= \lambda xy. xy\mathcal{F}$

ANAND $:= \lambda xy. xyx$

Do both LAND and ANAND perform logical and ?

Yes, if applied on **logical** values.

Example (LAND and ANAND on **non-logical** values)

LAND 2 3 = 0

ANAND 2 3 = 512

Conclusion

We should not perform logical operations on non-logical values.
To enforce this, we need **types** $\rightarrow$ Typed Lambda Calculus.

Why λ -calculus?

Definition (Semantics of sentence "Joe loves Marilyn")

Given by the predicate *loves(joe, marilyn)*

Definition (Compositionality for semantics)

When combining the semantics of the term *joe* (which is a person called Joe) and the semantics of the "*loves marilyn*" part, we should get *loves(joe, marilyn)*.

Why λ -calculus?

Definition (Structure of "Joe loves Marilyn")

$S = NP \oplus VP = NP \oplus (\text{Verb} \oplus \text{NP1})$, where
 $NP = \text{"Joe"}$, $\text{Verb} = \text{"loves"}$, $\text{NP1} = \text{"Marilyn"}$

Definition (Semantics of "Joe loves Marilyn")

- ▶ *joe*: person Joe
- ▶ *marilyn*: person Marilyn
- ▶ *loves*: $\lambda xy. \text{loves}(y, x)$
- ▶ *loves marilyn*: $\lambda x. \text{loves}(x, \text{marilyn})$ "arg loves Marilyn"
Combining operator: λ application
- ▶ *joe loves marilyn*: $\text{loves}(\text{joe}, \text{marilyn})$ "Joe loves Marilyn"

Combinatory logic

Definition (Combinators)

Combinators = special λ -expressions having no free variables.

Definition (Combinatory logic)

Combinatory logic = a version of λ -calculus which employs solely (a small set of) combinators instead of λ -expressions.

Why combinators?

Each λ -expression could be transformed into a combinator expression. Combinator reduction is simpler than λ reduction.

Combinatory logic

Definition (Combinator **I**: identity)

I := $\lambda x.x$, whose role is: **I** $M \equiv M$

Definition (Combinator **K**: building a constant)

K := $\lambda xy.x$, whose role is: **K** $M N \equiv M$

Definition (Combinator **S**: apply x over z and (y over z))

S := $\lambda xyz.xz(yz)$, whose role is: **S** $L M N \equiv L N (M N)$

Definition (Combinator **B**: apply x to "y applied to z")

B := $\lambda xyz.x(yz)$, whose role is: **B** $L M N \equiv L (M N)$

Combinatory logic

Combining combinators

Every combinator could be written using only **S** and **K**.

Example (Building combinator **I** from **S** and **K**)

I $\equiv$ **SKK**

$$\begin{aligned}(\mathbf{SKK})\ x &\equiv \mathbf{SKK}\ x \\ &\equiv \mathbf{K}\ x\ (\mathbf{K}\ x) \\ &\equiv x \\ &\equiv \mathbf{I}x\end{aligned}$$

Example (Building combinator **B** from **S** and **K**)

Show that **B** $\equiv$ **S(KS)K**

Combinatory logic

Combining combinators

Any λ -expression could be written using only combinators.

Example (Building λ -expressions using combinators)

$$\begin{aligned}\lambda p q. q \ p &\equiv \mathbf{S} \ (\mathbf{K} \ (\mathbf{S} \ \mathbf{I})) \ (\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I}) \\ &(\mathbf{S} \ (\mathbf{K} \ (\mathbf{S} \ \mathbf{I})) \ (\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I})) \ x \ y \\ &\equiv \mathbf{S} \ (\mathbf{K} \ (\mathbf{S} \ \mathbf{I})) \ (\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I}) \ x \ y \\ &\equiv (\mathbf{K} \ (\mathbf{S} \ \mathbf{I})) \ x \ ((\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I}) \ x) \ y \\ &\equiv (\mathbf{S} \ \mathbf{I}) \ (\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I} \ x) \ y \\ &\equiv \mathbf{I} \ y \ ((\mathbf{S} \ (\mathbf{K} \ \mathbf{K}) \ \mathbf{I} \ x) \ y) \\ &\equiv y \ ((\mathbf{K} \ \mathbf{K}) \ x \ (\mathbf{I} \ x)) \ y \\ &\equiv y \ (\mathbf{K} \ x \ y) \\ &\equiv y \ x\end{aligned}$$

Summary

1. Lambda Calculus is a minimalist universal programming language: it uses only the λ symbol, dot, parentheses and variable names.
2. It can express logical operations, numbers and arithmetic operations, conditionals, lists, recursivity etc.
3. Lambda Calculus sets the theoretical foundation for all functional programming languages, but it is also used for expressing formal semantics of natural languages.
4. Combinatory logic is a formalism equivalent to Lambda Calculus which is based on a set of special lambda expressions called combinators.
5. adding types to lambda calculus would help